
HiGym: Learning to Act in High-Dimensional Spaces

Anonymous Author(s)

Affiliation

Address

email

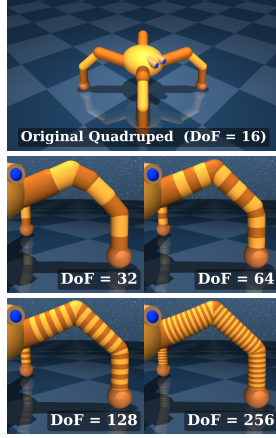
Abstract

1 Reinforcement learning (RL) performs strongly on continuous control bench-
2 marks, yet many real-world systems such as articulated robots, musculoskeletal
3 models, and dexterous hands require controlling tens to hundreds of actuators.
4 In this regime, action dimensionality can be a dominant bottleneck, but existing
5 benchmarks often entangle dimensionality with changes in objectives, morphol-
6 ogy, or contact dynamics, obscuring how algorithms truly scale. We introduce
7 HiGym, a high-dimensional continuous control benchmark built on DMControl.
8 HiGym comprises six environment families with multiple tasks and six embodi-
9 ment complexities. We isolate action-space scaling by keeping tasks and dynam-
10 ics as similar as possible while increasing degrees of freedom via a principled
11 embodiment-scaling procedure that recursively splits joints to reach target dimen-
12 sionalities. This yields matched task variants that differ primarily in action di-
13 mension. Across PPO, TD-MPC2, and TD-M(PC)², we find that model-based
14 methods degrade substantially less as dimensionality increases, preserving higher
15 sample-efficiency and asymptotic performance than model-free baselines. Among
16 model-based methods, we find that TD-M(PC)²'s policy-constraint loss provides
17 a consistent further gain at high DoF locomotion tasks by suppressing value over-
18 estimation that arises from a structural mismatch between the learned policy and
19 planner's behavior policy. **Code and videos:** <https://higym.github.io>.

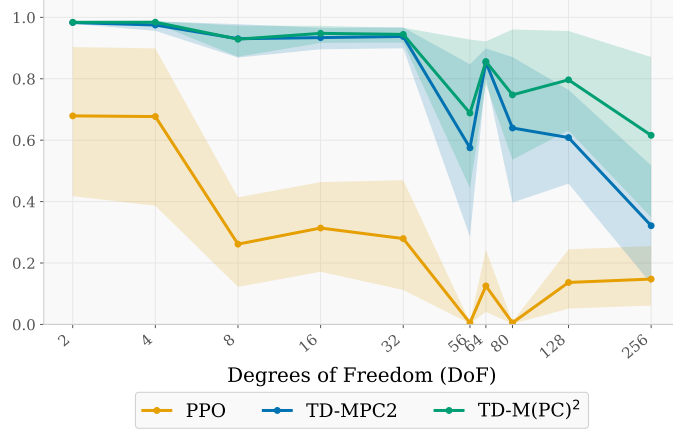
20 1 Introduction

21 Many real-world systems such as full-body humanoids, dexterous hands, musculoskeletal models,
22 and articulated robots require coordinating tens to hundreds of degrees of freedom (DoF) simulta-
23 neously. Reinforcement learning (RL) has become a reliable tool for continuous control at lower
24 dimensions [Mnih et al., 2013, Schulman et al., 2017, Haarnoja et al., 2018], but whether existing
25 algorithms scale as the action dimensionality grows remains a fundamental open question. This
26 question sits orthogonal to the more familiar scaling axes of model size, data, and compute [Hilton
27 et al., 2023, Hansen et al., 2024, Nauman et al., 2025]: it asks how the difficulty of the control
28 problem itself grows with its dimensionality, holding everything else fixed.

29 Existing benchmarks cannot answer this question cleanly. Recent suites cover musculoskeletal con-
30 trol [Caggiano et al., 2022], dexterous manipulation [Rajeswaran et al., 2018, Chen et al., 2022],
31 whole-body humanoid control [Sferrazza et al., 2024, Chernyadev et al., 2024], and large-scale
32 legged locomotion [Mittal et al., 2023], each reaching high actuator counts within a single fixed
33 embodiment. However, comparing across these suites simultaneously varies morphology, contact
34 dynamics, observation semantics, and reward shaping. Procedural morphology suites such as Uni-
35 mal and Metamorph [Gupta et al., 2021, 2022] also vary morphology, but with a different goal:
36 generalizing across embodiments rather than scaling within one. As a result, claims about high-DoF
37 performance conflate several factors at once, and the role of action dimensionality remains unclear.



(a) Quadruped leg complexity



(b) Aggregate IQM vs. DoF (all embodiment families)

Figure 1. HiGym benchmark overview. (a) Quadruped embodiment at increasing DoF via HiGym’s joint-splitting procedure. (b) Aggregate normalized IQM vs. DoF across all families and tasks: model-based methods degrade far less than PPO, and TD-M(PC)²’s policy-constraint loss yields a consistent further gain at high DoF.

We introduce **HiGym**, a continuous control benchmark with action dimensionality as its primary axis of variation. Built on top of the DeepMind Control Suite [Tassa et al., 2018], HiGym consists of six embodiment families (Quadruped, Cheetah, Walker, Finger, Reacher, and Humanoid CMU), each instantiated at multiple matched DoF variants via a principled *joint-splitting* procedure that recursively subdivides each kinematic chain. Three invariants are preserved across DoF: the total curvature budget per limb, the total limb mass, and the collision geometry of the embodiment. Tasks, reward functions, episode horizons, and termination criteria are held identical across DoF variants within a family. The result is a controlled scaling axis: a Quadruped at 512 DoF and at 16 DoF run the same task under near-identical dynamics, so any observed performance difference reflects the cost of acting in a higher-dimensional space rather than a change in objective. Figure 1 previews the suite and our main empirical finding.

We evaluate three RL algorithms across all six embodiment families of HiGym, covering both model-free and model-based methods: PPO [Schulman et al., 2017], TD-MPC2 [Hansen et al., 2024], and TD-M(PC)² [Lin et al., 2025], and **release code that reproduces our experiments at <https://higym.github.io>**. Our findings can be summarized as follows:

1. **Dimensionality is a universal bottleneck.** Every algorithm degrades with d , but the failure mode differs by class: PPO fails to learn beyond 32–64 DoF, and we trace this collapse to PPO’s clipped surrogate objective.
2. **Model-based methods are substantially more robust.** Both TD-MPC2 variants preserve sample efficiency and asymptotic performance at high DoF, suggesting that planning in a fixed-dimensional latent space might scale better than on-policy rollouts in the physical action space.
3. **The policy-constraint loss of TD-M(PC)² predominantly helps in locomotion families.** Its margin over TD-MPC2 widens monotonically with d in locomotion but is limited in manipulation, indicating that its benefit is family-dependent rather than uniform across high-DoF tasks.

2 Preliminaries

Problem setting. We consider continuous control formalized as a Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \rho, r, \gamma)$ with a d -dimensional action space $\mathcal{A} \subseteq \mathbb{R}^d$, and aim to learn a policy π maximizing the expected discounted return $J(\pi) = \mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^T \gamma^t r(s_t, a_t)]$. HiGym instantiates this setting at varying d while holding the reward r and episode length T fixed across DoF variants, so that differences in $J(\pi)$ reflect the difficulty of acting in a higher-dimensional space rather than changes in task specification. We evaluate three algorithms spanning the model-free and model-based spectrum; full technical details are deferred to the appendices.

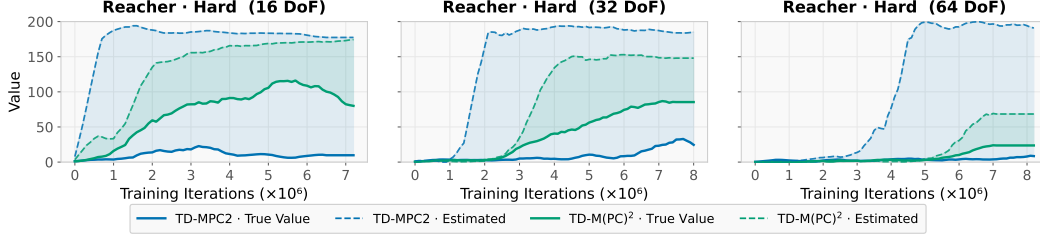


Figure 2. **Value overestimation on Reacher-Hard.** Estimated Q-value (dashed) vs. true Monte-Carlo return (solid) across DoF variants. TD-MPC2’s gap grows with DoF; TD-M(PC)² substantially reduces it by anchoring the policy to the MPPI distribution.

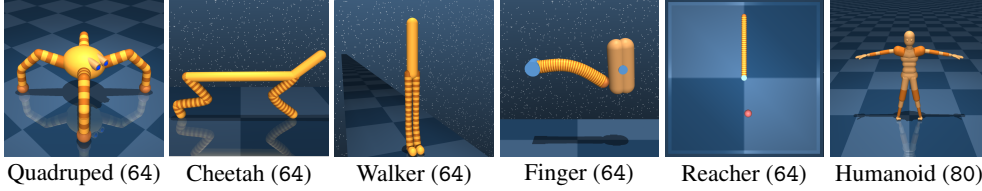


Figure 3. **HiGym embodiments at a shared high-DoF variant.** Each family is shown at its 64-DoF variant, except Humanoid CMU which is shown at its only scaled variant (80-DoF). The alternating colored stripes highlight the new distribution of joints.

70 **TD-MPC2** [Hansen et al., 2024] learns an *implicit world model* in a compact latent space and uses
 71 it for local trajectory optimization via MPPI planning [Williams et al., 2017]. Five components are
 72 jointly trained conditioned on a task embedding: an encoder, a latent dynamics model, a reward
 73 predictor, a terminal Q -function, and a policy prior that warm-starts the planner. At each decision
 74 step, MPPI optimizes a Gaussian action sequence over a horizon H using the learned reward and
 75 value, executes the first action, and replans (receding horizon).

76 **TD-M(PC)²** [Lin et al., 2025] targets a structural mismatch in TD-MPC2: the policy prior is trained
 77 by gradient ascent on Q and drifts away from the MPPI behavior distribution that populates the
 78 replay buffer, leaving Q to be evaluated at out-of-distribution state-action pairs. Figure 2 illustrates
 79 the resulting overestimation on Reacher-Hard, where the gap between TD-MPC2’s Q -estimates and
 80 the true Monte-Carlo returns widens with action dimension; the same trend holds across embodi-
 81 ments. TD-M(PC)² remedies this by augmenting the policy loss with a *policy-constraint* (PC) term,
 82 weighted by β , that regularizes π toward the planner distribution. Lin et al. [2025] report large gains
 83 on high-DoF humanoid tasks, but those experiments conflate dimensionality with embodiment, task
 84 semantics, and contact dynamics, a confound HiGym is designed to resolve.

85 **PPO** [Schulman et al., 2017] is a model-free on-policy RL algorithm that maximizes a clipped sur-
 86rogate objective to prevent destructively large policy updates. We use a parallelised implementation
 87 of PPO with 128 environments. See Appendix C for details.

88 3 HiGym: A Continuous Control Benchmark for DoF Scaling

89 No existing benchmark cleanly isolates action-space dimensionality as an experimental variable:
 90 prior work either fixes embodiment scale (precluding scaling analysis) or compares across unrelated
 91 morphologies that conflate dimensionality with contact dynamics, reward structure, and task seman-
 92 tics. Our proposed benchmark, **HiGym**, fills this gap with a principled joint-splitting procedure that
 93 varies d along a controlled axis while holding the rest of the task near-fixed.

94 3.1 Benchmark Design

95 HiGym builds on the DeepMind Control Suite [Tassa et al., 2018] and covers six environment fam-
 96 ilies: legged locomotion (Quadruped, Cheetah, Walker), whole-body humanoid control (Humanoid
 97 CMU), and planar manipulation (Finger, Reacher). See Figure 3 for a visualization of each em-
 98 bodiment family. Each family is instantiated at multiple DoF variants by recursively subdividing its
 99 kinematic chains. Table 1 summarizes the DoF range and tasks per family.

Table 1. HiGym benchmark suite. Six environment families with their base DoF, scaled DoF variants, and associated tasks.

Environment	Base DoF	Scaled DoFs	Tasks
Quadruped	16	16, 32, 64, 128, 256, 512	walk, run, escape, fetch
Cheetah	6	8, 16, 32, 64, 128, 256	run
Walker	6	8, 16, 32, 64, 128, 256	stand, walk, run
Finger	2	2, 4, 8, 16, 32, 64	spin, turn_easy, turn_hard
Reacher	2	2, 4, 8, 16, 32, 64	easy, hard
Humanoid CMU	56	80	stand, walk, run

Table 2. Per-family scaling laws. Range and gear exponents, mass conservation, and actuator type per family. *Exact*: budget mathematically preserved; *linear growth*: per-joint property constant. *Explicit* mass: each geom assigned limb mass/ n_{geoms} ; *Density*: MuJoCo default density $\times$ volume.

Family	Range α	Gear scaling	Mass	Actuator
Quadruped	1.0 (exact)	$(4/n)^{0.4} + \text{compensation}$	Explicit	Position servo
Cheetah	1.0 (exact)	$(3/n)^{0.1}$	Explicit	Motor
Walker	1.0 (exact)	Unscaled (linear growth)	Density	Motor
Finger	1.0 (exact)	$(2/n)^{0.7} / (2/n)^{0.85}$ at 64	Explicit	Motor
Reacher	1.0 (exact)	Unscaled (linear growth)	Density	Motor
Humanoid CMU	1.0 (exact)	50/50 split (exact)	Explicit	Motor

Given a base model with n actuated joints per kinematic chain, joint-splitting subdivides each chain by inserting intermediate bodies at equal arc-length positions along the original limb geometry. Three invariants are maintained across all families:

- **Total curvature budget.** Each new joint’s range is set so that the sum of joint ranges per limb equals that of the base model (range exponent $\alpha = 1.0$ in all families).
- **Total limb mass.** Geom masses are set so that the total limb mass matches the base model regardless of how many segments it is divided into.
- **Collision geometry.** Geom radii are preserved (no DoF-dependent shrinkage); intermediate bodies that are not end-effectors have collision disabled to prevent spurious self-contacts while keeping segment geometry consistent.

What changes across DoF? The action and observation spaces grow with d (each new joint adds a position and velocity component), but reward, episode length, and termination criteria remain fixed within a family, so any performance gap across DoF variants reflects the cost of higher-dimensional control rather than a change in task. The one budget that necessarily varies by embodiment is gear scaling: physical viability rules out a single uniform schedule, so per-family rules are listed in Table 2 (details in Appendices B.1–B.6). Geom radii are linearly interpolated from the base model when multiple radii are present.

3.2 Embodiments and Tasks

Quadruped. The base model has 16 DoF (4 joints per leg: 1 yaw and 3 pitch). Scaling subdivides each leg along the hip-to-toe arc, yielding variants at 32, 64, 128, 256, and 512 DoF. Actuators are position servos with gear scaled to maintain approximately constant peak contact torque as chains lengthen (Appendix B.1). Tasks: `walk` and `run` (forward locomotion with upright posture), `escape` (navigate hilly terrain), and `fetch` (retrieve a ball).

Cheetah. The base model is a planar biped (sagittal-plane runner) with 6 DoF (3 joints per leg). Scaling yields variants at 8, 16, 32, 64, 128, and 256 DoF. Pure torque conservation produces per-joint gear too weak for push-off at high DoF, so we use a mildly sub-linear gear schedule that lets total torque grow as $n^{0.9}$ (Appendix B.2); stiffness and damping are co-scaled with gear to preserve their ratios to motor authority. Task: `run`.

Walker. The base model is a planar biped with 6 DoF (3 joints per leg: hip, knee, ankle). Scaling yields variants at 8, 16, 32, 64, 128, and 256 DoF. Unlike Cheetah, gear ratios are left unscaled,

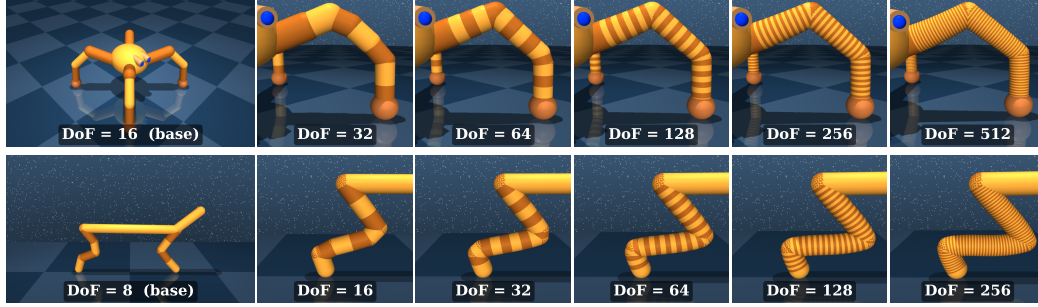


Figure 4. Quadruped and Cheetah embodiment scaling. Top row: Quadruped base body (16-DoF) and leg close-ups at 32, 64, 128, 256, and 512 DoF. Bottom row: Cheetah base body (8-DoF) and leg close-ups at 16, 32, 64, 128, and 256 DoF. Joint density increases while overall limb geometry, total reach, and total body mass are preserved.

so total leg torque grows linearly with n ; higher-DoF walkers are therefore stronger, introducing an inherent DoF-scaled control difficulty (Appendix B.3). Per-joint stiffness and damping are added to all variants to prevent high-DoF legs from collapsing passively under gravity. Tasks: stand, walk, and run.

Finger. The base model is a planar serial-chain manipulator with 2 DoF. Scaling extends the chain to 4, 8, 16, 32, and 64 DoF. Gravity is disabled by default, making the task purely contact-mediated. Per-joint gear tapers from base to tip and is scaled with chain length; the 64-DoF variant requires additional tuning to maintain spinner controllability (Appendix B.4). Tasks: spin (target spinner angular velocity), turn_easy, and turn_hard (rotate the spinner to a target angle, with two difficulty levels distinguished by target zone radius).

Reacher. The base model is a planar serial-chain arm with 2 DoF. Scaling extends the chain to 4, 8, 16, 32, and 64 DoF while holding total arm length fixed. The shoulder joint is left unlimited (free rotation), preserving the original reacher’s global-orientation degree of freedom. Non-shoulder joints share the curvature budget equally, and per-joint stiffness scales inversely with per-joint range to provide a passive restoring force that prevents chain curl at high DoF (Appendix B.5). Tasks: easy and hard (position the fingertip on a randomized target, with two difficulty levels distinguished by target radius).

Humanoid CMU. The base model is the 56-DoF CMU motion-capture skeleton from DMControl, extended to a single 80-DoF variant by inserting 12 intermediate bodies at the midpoints of major limb segments (thigh, shin, foot, upper arm, forearm, and shoulder, bilaterally). Joint ranges, gear ratios, and geom masses are split 50/50 between parent and child, exactly preserving the curvature, torque, and mass budgets (Appendix B.6). Tasks: stand, walk, and run.

4 Results

We evaluate three algorithms across HiGym: TD-MPC2, TD-M(PC)², and PPO. For each (algorithm, family, task, DoF) cell, we train 2 seeds for a maximum of 50M steps, stopping early if performance shows no measurable improvement over a 5M-step window. Asymptotic performance is the inter-quartile mean (IQM) of evaluation episode return over the last 10 evaluation episodes, normalized by the DMControl task maximum of 1000, so that per-task scores lie in $[0, 1]$. All uncertainty bands are 95% percentile-bootstrap CIs (10,000 resamples) over per-run asymptotic IQM, following [Agarwal et al., 2021].

Q1: How does action-space dimensionality affect asymptotic performance? Figure 5 plots asymptotic normalized IQM against action dimensionality for every (family, task) pair. PPO collapses sharply beyond 32–64 DoF in all locomotion families (Cheetah, Walker, Quadruped), approaching near-zero asymptotic reward at 128–256 DoF while both TD-MPC2 variants retain meaningful performance. The performance gap between model-based methods and PPO widens monotonically with DoF, confirming that a learned world model provides a structural advantage in high-dimensional action spaces. The gap between TD-MPC2 and TD-M(PC)² also widens with DoF,

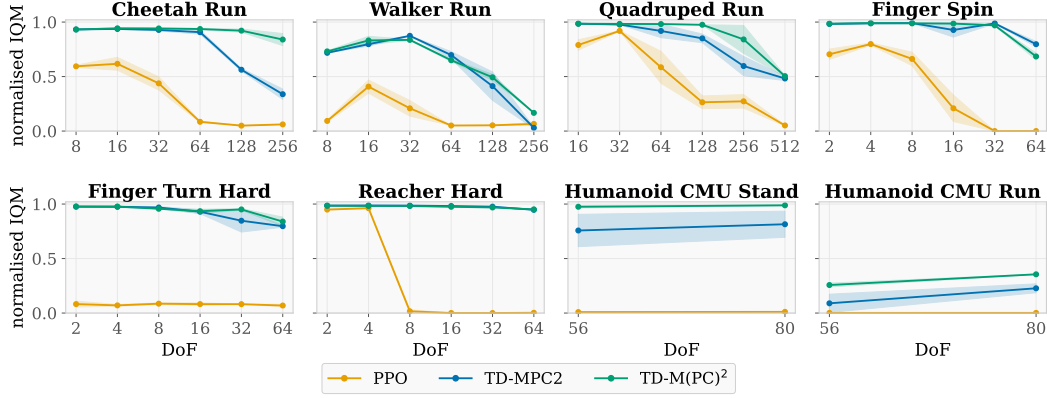


Figure 5. **Scaling curves across HiGym.** Asymptotic IQM (final 10% of training, normalized by task max 1000) vs. DoF on a log-2 scale, one panel per (family, task); shaded bands are 95% stratified-bootstrap CIs. Model-based methods degrade substantially less than PPO, with the gap widening monotonically with DoF.

167 consistent with the value-overestimation phenomenon described in Appendix A.2. Manipulation
 168 tasks (Finger, Reacher) show a milder DoF effect, likely because the underlying tasks remain effec-
 169 tively low-dimensional (*e.g.*, fingertip position) so added joints mostly add redundancy.

170 The rank-based view in Figure 14 (Appendix D.4) confirms this inference without reward-scale de-
 171 pendence. TD-MPC2 starts near rank 1.5 at low DoF but degrades toward rank 2.5 at 128–256 DoF
 172 as value overestimation intensifies, while TD-M(PC)² maintains rank 1 at 128–256 DoF, consistent
 173 with the PC-loss benefit.

174 **Q2: How do the three algorithms compare cumu-**
 175 **latively across HiGym?** Figure 6 shows the rank-1
 176 win rate: the fraction of (family, task, DoF) cells in
 177 which each algorithm achieves the highest asymptotic
 178 IQM. TD-M(PC)² wins 66.7% of cells overall, while
 179 PPO wins none. A threshold-free stochastic domi-
 180 nance comparison via the performance profile [Aga-
 181 wal et al., 2021] (Appendix D.3) further confirms that
 182 TD-M(PC)² stochastically dominates TD-MPC2 at
 183 every normalized return threshold, and both model-
 184 based methods dominate PPO. Together these results
 185 confirm that the ordering TD-M(PC)² $\succ$ TD-MPC2
 186 $\succ$ PPO is consistent and statistically robust across the
 187 full benchmark.

188 **Q3: Does the policy-constraint loss benefit all**
 189 **task types equally?** Figure 7 breaks the ranking
 190 down by embodiment family. TD-M(PC)² achieves
 191 rank 1.00 in Quadruped and Humanoid-CMU, where value overestimation is most severe (see Ap-
 192 pendix A.2). Notably, TD-MPC2 is tied with or slightly ahead of TD-M(PC)² in Finger and Reacher.

193 The per-family breakdown reveals a consistent pattern: the policy-constraint loss delivers clear gains
 194 across all locomotion families (Cheetah, Walker, Quadruped, Humanoid-CMU) but provides little
 195 to no benefit in the planar manipulation families (Finger, Reacher). While the precise cause is not
 196 fully understood, we hypothesize that MPPI coverage of the action space is substantially sparser in
 197 locomotion than in manipulation. Locomotion requires coordinating multiple coupled limbs simul-
 198 taneously across a high-dimensional joint space, so any given MPPI rollout explores only a small
 199 fraction of feasible trajectories and leaves large regions of action space uncovered. Planar serial-
 200 chain manipulation tasks, by contrast, provide dense and localized contact feedback that naturally
 201 constrains the effective action space to a compact region even at high DoF, limiting how far the pol-
 202 icy can drift from the planner distribution. When MPPI coverage is dense, the policy prior remains

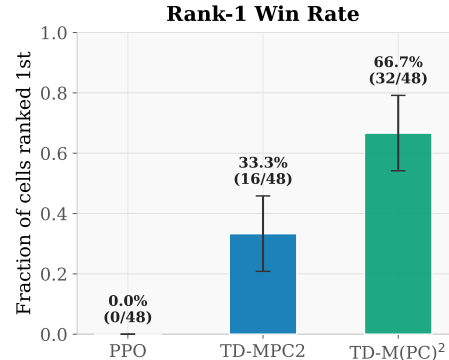


Figure 6. **Rank-1 win rate across HiGym.** Fraction of (family, task, DoF) cells in which each algorithm achieves the highest asymptotic IQM. TD-M(PC)² wins the majority of cells.

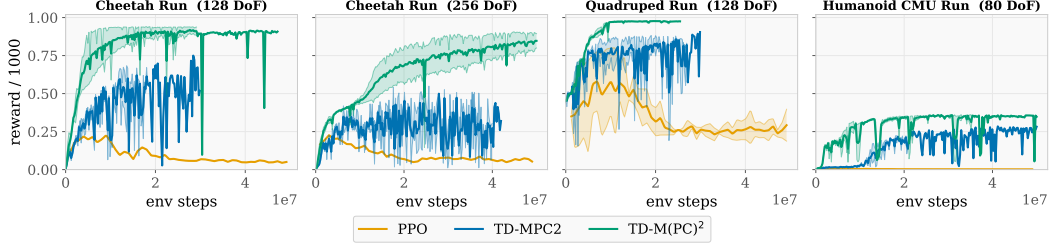


Figure 8. **Per-DoF learning curves.** Training return vs. environment steps for Cheetah-run, Quadruped-run, and Humanoid-CMU run at representative high-DoF variants; min-max seed bands with bold mean. PPO fails to learn; TD-MPC2 learns with high variance; TD-M(PC)² achieves the highest and most stable returns.

close to the behavior distribution without explicit regularization and the constraint provides diminishing marginal returns. On the other hand, when coverage is sparse, as in coupled locomotion, policy drift is large and value overestimation accumulates, making the constraint essential.

This finding is consistent with the original TD-M(PC)² evaluation [Lin et al., 2025], which demonstrates that the constraint is most beneficial in high-DoF humanoid locomotion. Our HiGym results extend this observation to a broader set of locomotion morphologies along with studying manipulation tasks under controlled DoF scaling, indicating that in addition to dimensionality, task type also governs whether the policy constraint provides a meaningful benefit.

Q4: How does action dimensionality affect sample efficiency?

Aggregate asymptotic metrics can obscure training dynamics; Figure 8 shows per-seed training curves for representative high-DoF settings spanning locomotion (Cheetah-run, Quadruped-run) and whole-body control (Humanoid-CMU), revealing qualitatively distinct failure modes for each method. PPO’s curves flatten almost immediately at 128–256 DoF in locomotion and remain near zero throughout training. TD-MPC2 continues to improve but with high inter-seed variance, exhibiting large reward oscillations consistent with value overestimation. TD-M(PC)² learns steadily with substantially lower variance, indicating that the PC loss stabilizes the training signal rather than merely shifting the asymptotic level. On Humanoid-CMU’s run task at 80 DoF, PPO remains flat while both TD-MPC2 variants learn to walk, with TD-M(PC)² achieving a faster gait. TD-M(PC)² also unlocks higher overall rewards for Cheetah and Quadruped at 128 and 256 DoF. The same trend appears quantitatively in Appendix D.1 (Figure 12), where the steps-to-75%-return metric scales sharply with DoF for PPO and the PC loss yields a sample-efficiency gain that widens with dimensionality. Refer to Appendix D.2 for a complete set of training curves.

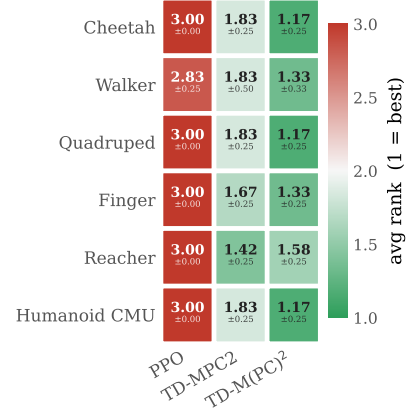


Figure 7. **Mean rank per embodiment family.** Color: green = rank 1, red = rank 3; values averaged across all tasks, DoFs, and seeds within each family.

Q5: What failure modes emerge at high DoF, and why does PPO collapse structurally? The convergent evidence above motivated us into probing the structural cause of PPO’s failure. From the scaling curves (Figure 5), PPO’s performance is competitive below 32–64 DoF and approaches near-zero above it, while both TD-MPC2 variants degrade smoothly. The per-DoF learning curves (Figure 8) reveal that PPO’s curves do not flatten gradually, instead they stagnate or actively deteriorate within the first evaluation windows at 128–256 DoF, ruling out slow convergence as an explanation.

We find that the root cause lies in how PPO’s clipped surrogate objective scales with d . For a factorised Gaussian policy, the joint probability ratio is the product across all d dimensions:

$$r_{\text{joint}}(\theta) = \prod_{i=1}^d \frac{\pi_{\theta}(a_i | s)}{\pi_{\theta_{\text{old}}}(a_i | s)}. \quad (1)$$

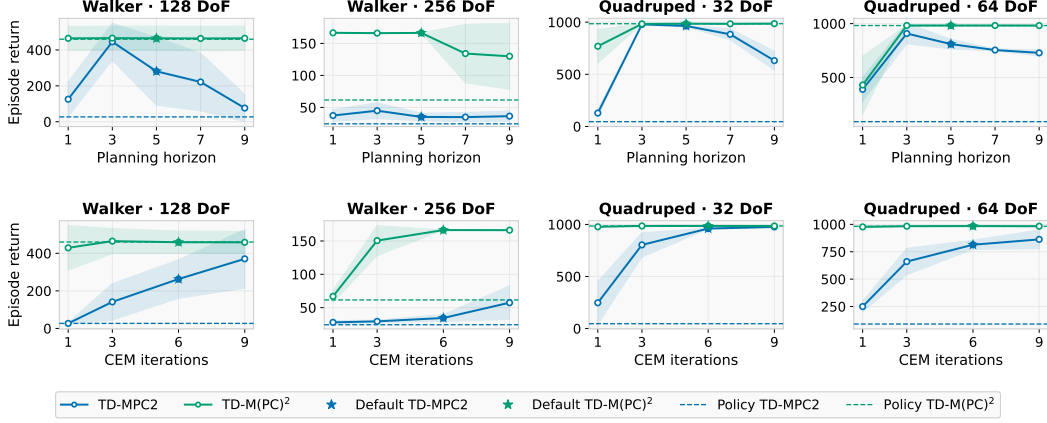


Figure 9. Planning budget sensitivity. Episode return vs. planning horizon (top, CEM= 6) and CEM iterations (bottom, horizon= 3) for Walker-128, Walker-256, Quadruped-32, and Quadruped-64 run tasks. Colored ★ marks each algorithm’s default operating point (horizon= 3, CEM= 6); colored dashed line is the corresponding policy-only baseline π_θ (no planning). TD-M(PC)² is substantially flatter than TD-MPC2, and its policy baseline is markedly higher.

PPO clips this *scalar* joint ratio to $[1-\varepsilon, 1+\varepsilon]$, which is a single one-dimensional constraint on a d -dimensional quantity. At large d this clip is geometrically degenerate: per-dimension factors can take large opposing values that cancel in the product, leaving the joint ratio inside the clip while individual coordinates move arbitrarily far from π_{old} . The scalar joint clip therefore offers no effective per-dimension control once d is large, permitting single updates that move individual action coordinates far enough to destroy previously learned behavior.

Q6: How sensitive are model-based methods to planning budget reductions? Figure 9 shows how episode return varies with planning horizon and CEM iteration count at four representative (family, DoF) cells. TD-M(PC)² is strikingly robust to both budget dimensions: at Walker-128 DoF it holds ~ 500 return across the full horizon sweep (1–9), while TD-MPC2 collapses to ~ 100 at short horizons. At Quadruped-32 and 64 DoF, TD-M(PC)² sustains >950 return at every budget setting except horizon 1, whereas TD-MPC2 is substantially more sensitive to horizon length and requires several CEM iterations to recover similar performance.

A second effect is visible in the per-algorithm policy baselines (dashed lines): at all DoFs, TD-M(PC)²’s policy π_θ alone already achieves substantial return, whereas TD-MPC2’s policy operates near zero without planning support. This indicates that the PC loss produces a qualitatively stronger standalone policy prior in addition to a better planner. TD-M(PC)²’s budget robustness therefore has two complementary sources: a higher-quality policy that partially substitutes for planning when the budget is cut, and value-guided planning that further improves over the policy when budget permits. In contrast, TD-MPC2 relies almost entirely on the planner, making it fragile when horizon or iterations are reduced. Practically, this means high-DoF deployments using TD-M(PC)² can use shorter horizons or fewer CEM iterations without meaningful performance loss, providing a direct computational saving. Full budget sensitivity curves across all DoF variants are provided in Appendix D.7.

5 Related Work

Continuous control benchmarks. The DeepMind Control Suite [Tassa et al., 2018] has long served as the canonical testbed for RL for continuous control, alongside the Gym/MuJoCo locomotion tasks and the broader `dm_control` stack. These suites established the standard low-to-mid DoF regime on which the algorithms we evaluate were designed. Recent benchmarks push toward substantially higher complexity along several distinct axes: legged locomotion at scale in Isaac Gym and Isaac Lab [Makoviychuk et al., 2021, Mittal et al., 2023], musculoskeletal control in MyoSuite [Caggiano et al., 2022], dexterous manipulation in Adroit, Bi-DexHands, and DexArt [Rajeswaran et al., 2018, Chen et al., 2022, Bao et al., 2023], broader robot-learning suites such as RoboHive [Kumar et al.,

275 2023], and whole-body humanoid control suites such as HumanoidBench and BiGym [Sferrazza
 276 et al., 2024, Chernyadev et al., 2024]. Each benchmark reaches tens or hundreds of actuators within
 277 a single fixed embodiment. However, moving between them simultaneously changes morphology,
 278 contact dynamics, observation semantics, and reward shaping, so any performance gap is jointly
 279 attributable to all of these factors rather than to action dimensionality alone. Closer in spirit, pro-
 280 cedural morphology benchmarks such as Unimal and Metamorph [Gupta et al., 2021, 2022] vary
 281 morphology, but with the goal of generalizing *across* embodiments rather than studying scaling
 282 within an embodiment family. In contrast, HiGym is constructed with action dimensionality as the
 283 principal axis of variation: our proposed joint-splitting procedure produces matched task variants
 284 that share rewards and approximately preserve dynamics, so observed differences across DoF can
 285 be directly attributed to the action dimension itself.

286 **Scaling and the curse of action dimensionality.** Empirical scaling laws [Hilton et al., 2023, Hansen
 287 et al., 2024, Nauman et al., 2025] relate sample complexity to model size, data, or compute while
 288 holding the environment or dataset fixed; the orthogonal question of how performance scales with
 289 the *action dimension* of the task itself has received little systematic study, despite clear theoretical
 290 reasons to expect it to matter. Sampling-based planners such as CEM and MPPI [Williams et al.,
 291 2017] incur exponential coverage decay in d at fixed sample budget, and policy gradient methods
 292 inherit a per-dimension exploration cost through their action-space noise distribution. The standard
 293 response in the high-DoF literature has been to change the policy parameterization rather than to
 294 diagnose the scaling itself: factored or autoregressive action heads [Vinyals et al., 2019, Metz et al.,
 295 2017], latent-action and synergy-based controllers exploiting muscle co-activation [Caggiano et al.,
 296 2022, Chiappa et al., 2023], and hierarchical decompositions [Nachum et al., 2018] all aim to reduce
 297 the effective control dimension. While these methods improve absolute performance, they do not
 298 answer the prior question: *at fixed algorithm and parameterization, how does sample efficiency*
 299 *degrade with d ?* By holding morphology and reward fixed across DoF, HiGym shows that all RL
 300 algorithms degrade with increasing d but that model-based methods generally are more robust.

301 **Model-based RL and value overestimation.** Model-based RL has produced strong continuous
 302 control results via latent dynamics models such as Dreamer [Hafner et al., 2020, 2023] and TD-
 303 MPC [Hansen et al., 2022, 2024], as well as Dyna-style hybrids such as MBPO [Janner et al., 2019].
 304 Orthogonally, value-based RL has long contended with value overestimation driven by the critic
 305 being queried at actions outside the support of the data [Thrun and Schwartz, 1993, Hasselt et al.,
 306 2016]. For this reason, offline RL literature tends to anchor the learned policy to the behavior dis-
 307 tribution so that out-of-distribution actions are never queried [Kumar et al., 2020, Fujimoto et al.,
 308 2019, Fujimoto and Gu, 2021]. TD-M(PC)² [Lin et al., 2025] observes that TD-MPC2 suffers the
 309 same failure *online*: the MPPI planner generates the replay buffer, but the parametric policy prior is
 310 trained by gradient ascent on Q , so π drifts off the planner’s support and the critic is evaluated on
 311 unseen actions. Their remedy ports the offline anchoring recipe to this setting, using the MPPI pos-
 312 terior as the behavior reference. The original evaluation, however, co-varies action dimensionality
 313 with embodiment, contact regime, and reward, leaving the source of the gain ambiguous. HiGym
 314 removes these confounders: by varying only d within each family, we confirm the effect is genuinely
 315 action-dimensional and show that the policy–planner divergence grows monotonically with d .

316 6 Limitations and Future Work

317 We propose HiGym, a comprehensive benchmark for high-DoF continuous control that isolates ac-
 318 tion space dimensionality from confounds such as changes in contact dynamics or reward shaping.
 319 However, HiGym currently evaluates only three algorithms. Expanding to broader algorithm fam-
 320 ilies, including model-based methods with explicit action-space decomposition, hierarchical con-
 321 trollers, or latent-space diffusion policies would strengthen the empirical conclusions. Additionally,
 322 a preliminary curriculum experiment that warm-starts a high-DoF policy from a trained low-DoF
 323 policy of the same family (Appendix D.8) yields a non-trivial sample-efficiency gain at 256 DoF.
 324 We thus view a systematic study of cross-DoF transfer as a promising direction for future work.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2021. Curran Associates Inc.
- Chen Bao, Helin Xu, Yuzhe Qin, and Xiaolong Wang. DexArt: Benchmarking generalizable dexterous manipulation with articulated objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Vittorio Caggiano, Huawei Wang, Guillaume Durandau, Massimo Sartori, and Vikash Kumar. Myosuite – a contact-rich simulation suite for musculoskeletal motor control, 2022.
- Yuanpei Chen, Yaodong Yang, Tianhao Wu, Shengjie Wang, Xidong Feng, Jiechuang Jiang, Stephen Marcus McAleer, Hao Dong, Zongqing Lu, and Song-Chun Zhu. Towards human-level bimanual dexterous manipulation with reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.
- Nikita Chernyadev, Nicholas Backshall, Xiao Ma, Yunfan Lu, Younggyo Seo, and Stephen James. BiGym: A demo-driven mobile bi-manual manipulation benchmark. *arXiv preprint arXiv:2407.07788*, 2024.
- Alberto Silvio Chiappa, Alessandro Marin Vargas, Ann Huang, and Alexander Mathis. Latent exploration for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. *ArXiv*, abs/2106.06860, 2021.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning (ICML)*, 2019.
- Agrim Gupta, Silvio Savarese, Surya Ganguli, and Li Fei-Fei. Embodied intelligence via learning and evolution. *Nature Communications*, 12(1):5721, 2021. doi: 10.1038/s41467-021-25874-z.
- Agrim Gupta, Linxi Fan, Surya Ganguli, and Li Fei-Fei. MetaMorph: Learning universal controllers with transformers. In *International Conference on Learning Representations (ICLR)*, 2022.
- Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, G. Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, P. Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *ArXiv*, abs/1812.05905, 2018.
- Danijar Hafner, Timothy P. Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *ArXiv*, abs/1912.01603, 2020.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, 2022.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- H. V. Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *Aaai*, 2016.
- Jacob Hilton, Jie Tang, and John Schulman. Scaling laws for single-agent reinforcement learning. *arXiv preprint arXiv:2301.13442*, 2023.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *ArXiv*, abs/1906.08253, 2019.

370 Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline
371 reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

372 Vikash Kumar, Rutav Shah, Gaoyue Zhou, Vincent Moens, Vittorio Caggiano, Jay Vakil, Abhishek
373 Gupta, and Aravind Rajeswaran. RoboHive: A unified framework for robot learning. In *Advances
374 in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

375 Haotian Lin, Pengcheng Wang, Jeff Schneider, and Guanya Shi. Td-m (pc)²: Improving temporal
376 difference mpc through policy constraint. *arXiv preprint arXiv:2502.03550*, 2025.

377 Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin,
378 David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High
379 performance GPU-based physics simulation for robot learning. In *Advances in Neural Informa-
380 tion Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

381 Luke Metz, Julian Ibarz, Navdeep Jaitly, and James Davidson. Discrete sequential prediction of
382 continuous actions for deep RL. *arXiv preprint arXiv:1705.05035*, 2017.

383 Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan,
384 Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State,
385 Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot
386 learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

387 Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wier-
388 stra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint
389 arXiv:1312.5602*, 2013.

390 Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforce-
391 ment learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

392 Michal Nauman, Marek Cygan, Carmelo Sferrazza, Aviral Kumar, and Pieter Abbeel. Bigger, regu-
393 larized, categorical: High-capacity value functions are efficient multi-task learners. *arXiv preprint
394 arXiv:2505.23150*, 2025.

395 Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel
396 Todorov, and Sergey Levine. Learning Complex Dexterous Manipulation with Deep Reinforce-
397 ment Learning and Demonstrations. In *Proceedings of Robotics: Science and Systems (RSS)*,
398 2018.

399 John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy
400 optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

401 Carmelo Sferrazza, Dun-Ming Huang, Xingyu Lin, Youngwoon Lee, and Pieter Abbeel. Humanoid-
402 Bench: Simulated humanoid benchmark for whole-body locomotion and manipulation. *arXiv
403 preprint arXiv:2403.10506*, 2024.

404 Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Bud-
405 den, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Ried-
406 miller. Deepmind control suite, 2018.

407 Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement
408 learning. In *Proceedings of the Fourth Connectionist Models Summer School*, 1993.

409 Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Juny-
410 oung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan
411 Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou,
412 Max Jaderberg, Alexander S. Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David
413 Budden, Yury Sulsky, James Molloy, Tom L. Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff,
414 Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom
415 Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver.
416 Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):
417 350–354, 2019.

418 Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and
419 Evangelos A. Theodorou. Information theoretic mpc for model-based reinforcement learning.
420 In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721,
421 2017.

A Algorithm Details

A.1 TD-MPC2

TD-MPC2 [Hansen et al., 2024] is a model-based RL algorithm that performs local trajectory optimization (planning) in the latent space of a learned *implicit world model*. We use the official reference implementation at <https://github.com/nicklashansen/tdmpc2> and adopt the default hyperparameters released by the authors without modification. During training, it learns the world model using the dataset generated from planning. Five components are trained jointly conditioned on a task embedding:

- **Encoder** h : latent state $z_t = h(s_t, \mathbf{e})$, mapping observations to a fixed-dimensional latent space normalized by SimNorm.
- **Dynamics** d : next latent prediction $\hat{z}'_t = d(z_t, a_t, \mathbf{e})$.
- **Reward** R : scalar prediction $\hat{r}_t = R(z_t, a_t, \mathbf{e})$.
- **Terminal Value** Q : discounted-return estimate $\hat{q}_t = Q(z_t, a_t, \mathbf{e})$, with targets $q_t \doteq r_t + \gamma Q(z'_t, \pi(z'_t, \mathbf{e}))$, where $\bar{Q}$ is an exponential moving-average of Q .
- **Policy prior** π : action $\hat{a}_t = \pi(z_t, \mathbf{e})$, which predicts the action a^* that maximises Q .

All components except the policy prior are trained by minimizing the following *world model loss* over a horizon H with geometric weighting λ^t [Hansen et al., 2024]:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')_{0:H} \sim \mathcal{B}} \left[\sum_{t=0}^H \lambda^t \left(\|d(z_t, a_t, \mathbf{e}) - \text{sg}(h(s'_t, \mathbf{e}))\|_2^2 + \text{CE}(\hat{r}_t, r_t) + \text{CE}(\hat{q}_t, q_t) \right) \right], \quad (2)$$

where $\text{sg}(\cdot)$ is the stop-gradient operator and CE denotes cross-entropy loss over a discrete, log-transformed return distribution. The policy prior is trained separately as a stochastic maximum-entropy policy:

$$\mathcal{L}_p(\theta) = \mathbb{E}_{(s,a)_{0:H} \sim \mathcal{B}} \left[\sum_{t=0}^H \lambda^t (-\alpha Q(z_t, \pi(z_t, \mathbf{e}), \mathbf{e}) - \beta \mathcal{H}(\pi(\cdot | z_t, \mathbf{e}))) \right], \quad (3)$$

updating α using moving statistics while keeping β constant.

At decision time, TD-MPC2 runs K iterations of MPPI [Williams et al., 2017] to solve:

$$\mu^*, \sigma^* = \arg \max_{\mu, \sigma} \mathbb{E}_{\mathbf{a}_{t:t+H} \sim \mathcal{N}(\mu, \sigma^2)} \left[\gamma^H Q(z_{t+H}, \pi(z_{t+H}, \mathbf{e}), \mathbf{e}) + \sum_{h=t}^{H-1} \gamma^h R(z_h, a_h, \mathbf{e}) \right], \quad (4)$$

bootstrapping with Q beyond the horizon. Only the first action $a_t \sim \mathcal{N}(\mu_t^*, \sigma_t^{*2})$ is executed, after which a new optimization is solved for step $t+1$ (receding horizon).

A.2 Value Overestimation in High-Dimensional Action Spaces

A structural mismatch exists within the TD-MPC2 training loop: at every environment step the *planner* draws actions from the MPPI posterior μ_k , which populates the replay buffer, but the *policy* π is independently optimized by gradient ascent on Q [Lin et al., 2025]. As π drifts away from μ_k , it queries Q at action-state pairs never visited during planning, where Q is poorly calibrated. Figure 10 directly confirms this pattern on HiGym for Cheetah-run.

Lin et al. [2025] formalize this as Theorem 4.2: the total-variation divergence between any two policies is bounded below by their performance gap,

$$\max_s D_{\text{TV}}(\pi'(\cdot | s) \| \pi(\cdot | s)) \geq \frac{(1-\gamma)^2}{2 R_{\max}} |J^\pi - J^{\pi'}|, \quad (5)$$

so any divergence from the behavior policy μ_k directly lower-bounds the sub-optimality of the current iterate. Theorem 4.1 further shows that TD-MPC planning errors accumulate across iterations: the gap δ_k grows via a multiplicative factor $(1 + \gamma^H) \delta_{k-1}$, amplifying early estimation errors over training.

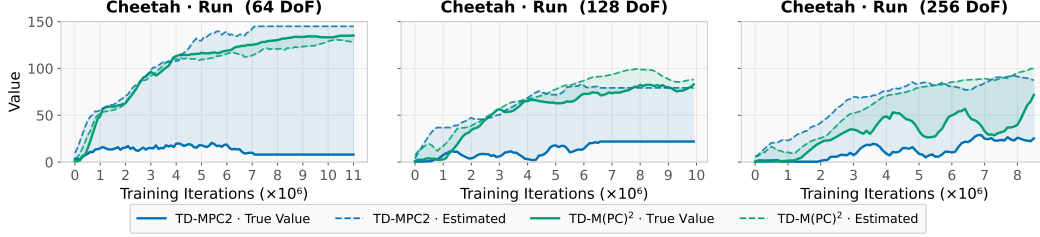


Figure 10. **Value overestimation on Cheetah-run across DoF.** Estimated Q-value (dashed) and true Monte-Carlo return (solid) at 64, 128, and 256 DoF; shaded region marks the overestimation gap. TD-MPC2’s gap widens monotonically with DoF; TD-M(PC)² substantially reduces it by anchoring the policy to the MPPI distribution.

Empirically, Lin et al. [2025] measure Q -value overestimation errors of **231–2159%** on high-dimensional humanoid tasks (61 DoF), compared to only $\sim 15\%$ on low-dimensional tasks, quantifying exactly the dimension-dependent amplification of this failure mode.

Figure 11 quantifies this trend across DoF on HiGym by aggregating overestimation ratios across all families and tasks. The aggregate directly validates the dimension-dependent amplification predicted by Lin et al. [2025]: TD-MPC2 overestimation grows monotonically from **13%** at 2 DoF to **396%** at 8 DoF and **1169%** at 80 DoF and escalates sharply to **7832%** at 256 DoF. We omit 512 DoF from this analysis entirely, as none of the algorithms acquired useful behavior or well-calibrated value functions at that dimensionality. TD-M(PC)² suppresses overestimation by an order of magnitude across the entire DoF range, never exceeding **35%** even at 256 DoF, by anchoring the policy to the MPPI planner distribution and eliminating out-of-distribution Q queries. This mitigation of value overestimation is the primary cause of the performance advantages reported in Section 4.

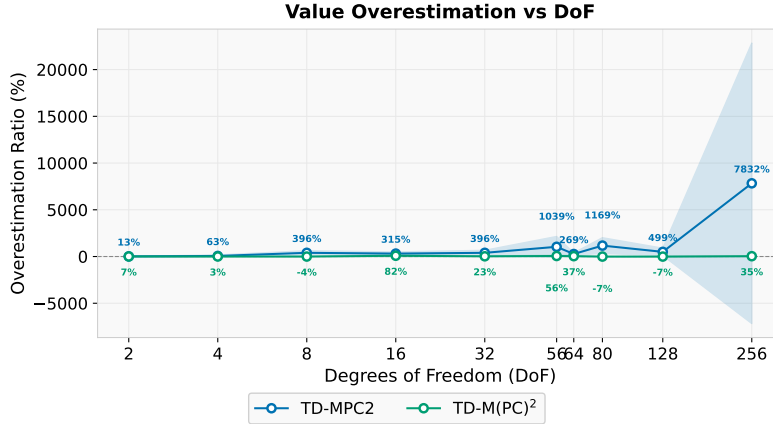


Figure 11. **Aggregate value overestimation ratio vs. DoF.** Mean overestimation (%) across all HiGym families and tasks (final 20% of training). TD-MPC2 escalates monotonically to **7832%** at 256 DoF; TD-M(PC)² never exceeds **35%**, confirming the policy-constraint loss directly suppresses this failure mode.

A.3 TD-M(PC)²: Policy-Constrained TD-MPC2

TD-M(PC)² [Lin et al., 2025] remedies the policy–planner divergence by augmenting the policy objective with a *distributional regularizer* toward the MPPI behavior policy. The MPPI posterior μ_k at each step is stored in the replay buffer alongside the transition, making it available as a behavior-cloning target at training time.

Formally, TD-M(PC)² solves a *Constrained Policy Improvement* (CPI) problem at each update:

$$\pi_{k+1} = \arg \max_{\pi} \mathbb{E}_{a \sim \pi(\cdot|s)} [Q_k(s, a)] \quad \text{s.t.} \quad D_{\text{KL}}(\pi_{k+1} \parallel \mu_k) \leq \varepsilon, \quad (6)$$

where μ_k is the current buffer behavior policy. The Lagrangian relaxation of (6) yields the practical policy loss [Lin et al., 2025]:

$$\mathcal{L}_\pi = -\mathbb{E}_{a \sim \pi}[Q(s, a) - \alpha \log \pi(a | s) + \beta \log \mu(a | s)], \quad (7)$$

where the first term maximises the Q-value, the α term maintains a maximum-entropy policy (as in standard SAC-style training), and the β term is the *Policy Constraint*: it rewards the policy for remaining close to the planner distribution μ , encouraging it to select in-domain actions. Setting $\beta = 0$ recovers the TD-MPC2 policy loss (3) exactly.

In our implementation we closely follow the reference codebase of Lin et al. [2025] at https://github.com/DarthUtopian/tdmpc_square_public and adopt its released hyperparameters unchanged on top of the TD-MPC2 defaults. When the PC loss is enabled ($\beta > 0$), the running Q-value scale is updated *before* computing $\mathcal{L}_\pi$ (pre-backward update), matching the reference timing. The PC term is additionally gated by a threshold $\tau_{\text{scale}} = 2.0$: it is activated only once the Q-scale exceeds this value, ensuring the Q-function has stabilized before the constraint engages. In practice, the prior log-probability is normalized by the action dimension d and a reference dimension of 61 (the humanoid DoF used in the original work), giving an effective prior coefficient of $\beta_{\text{eff}} = \beta \cdot d / 61$. This normalisation keeps the constraint strength proportional to the number of dimensions being constrained, which is relevant in our high-DoF setting where d ranges up to 256. We use $\beta = 1$ throughout; following [Lin et al., 2025] which reports low sensitivity across $\beta \in [0.05, 1.0]$.

B Embodiment Details

B.1 Quadruped: Design Details

The quadruped is a four-legged torso-driven locomotor based on the DMControl quadruped model. The base variant has 16 DoF (4 joints per leg: 1 yaw + 3 pitch). Scaling proceeds by subdividing each leg into more segments along the original hip-to-toe arc, yielding variants at 32, 64, 128, 256, and 512 DoF.

Joint ranges. The single yaw joint per leg is held fixed at $\pm 50^\circ$ across all sizes. Only the pitch joints scale, with each pitch range shrinking as $(3/n_{\text{pitch}})^{1.0}$ to preserve the total curvature budget per limb.

Actuators and gear. Actuators are position servos rather than torque motors. The servo gain is set to $1000 \cdot (4/n_{\text{joints}})^{0.4}$ with a per-joint compensation factor that matches the control range to the joint range in radians, keeping the maximum achievable torque approximately constant across DoF counts. Joint damping grows as $(n_{\text{joints}}/4)^{0.5}$ to maintain numerical stability of the longer kinematic chains.

Mass and geometry. Leg mass is held constant by distributing the original limb mass equally among segments. Geom radii are interpolated from the original hip-to-ankle profile ($0.08 \rightarrow 0.065 \rightarrow 0.055$ m) with no DoF-dependent shrinkage. Only toe spheres participate in ground contact; intermediate capsules are collision-filtered to prevent spurious self-contacts while keeping segment geometry consistent.

B.2 Cheetah Gear Scaling: Why $\alpha = 0.1$ and Why Uniform

Background. The cheetah has two legs (back and front), each with 3 joints at the 6-DoF baseline. Scaling to n joints per leg requires choosing *two* per-joint shrinkage exponents: a range exponent α_{range} for joint limits (per-joint range $\propto (3/n)^{\alpha_{\text{range}}}$) and a gear exponent α_{gear} for the gear/stiffness/damping triple (per-joint gear $\propto (3/n)^{\alpha_{\text{gear}}}$, applied uniformly to all three – see below). At $\alpha_{\text{gear}}=0$ (“walker strategy”) per-joint gear is unscaled and total torque per leg grows linearly in n ; at $\alpha_{\text{gear}}=1$ gear scales inversely with n so total torque is exactly conserved. At $\alpha_{\text{range}}=1$ total per-leg curvature is exactly conserved; lower values let the curvature budget grow with n . The choice of $(\alpha_{\text{range}}, \alpha_{\text{gear}})$ proved highly non-trivial: we performed a systematic sweep across the grid $\{0.3, 0.4, 0.5, 0.55, 0.6, 0.7, 0.8, 1.0\}^2$, and only the corner $(\alpha_{\text{range}}, \alpha_{\text{gear}})=(1.0, [0, 0.1])$ avoided every failure mode documented below. The final adopted setting is $(\alpha_{\text{range}}, \alpha_{\text{gear}})=(1.0, 0.1)$.

524 **Why α must be applied to gear, stiffness, and damping together.** The original cheetah enforces
525 two passive-dynamics ratios on every joint:

$$\begin{aligned} \text{stiffness/gear} &= 2.0 \quad (\text{restoring spring at joint limit} = 2 \times \text{motor max torque}), \\ \text{damping/gear} &= 0.05 \quad (\text{max joint velocity } v_{\max} = 20 \text{ rad/s}). \end{aligned}$$

526 If gear is scaled by $(3/n)^\alpha$ while stiffness and damping are left unchanged, the stiffness-to-gear
527 ratio grows as $(n/3)^\alpha$ with DoF: at 64 joints/leg the spring torque outweighs motor authority by a
528 factor of ~ 4 at $\alpha = 0.4$, effectively freezing all joints. Our implementation avoids this by sharing a
529 single `gear_scale` $= (3/n)^\alpha$ factor across all three quantities. This guarantees $\text{stiffness/gear} = 2.0$
530 and $\text{damping/gear} = 0.05$ exactly at every DoF count, verified numerically in Table 3.

Table 3. Cheetah hip-joint ratios under $\alpha = 0.1$ (v5.1). All ratios are preserved to four decimal places; $v_{\max} = 20$ rad/s is invariant.

DoF	joints/leg	hip gear	hip stiffness	stiff/gear	damp/gear
8	4	116.6	233.2	2.000	0.0500
16	8	108.8	217.6	2.000	0.0500
32	16	101.5	203.0	2.000	0.0500
64	32	94.7	189.4	2.000	0.0500
128	64	88.4	176.8	2.000	0.0500

531 **Sweep over the $(\alpha_{\text{range}}, \alpha_{\text{gear}})$ grid.** We evaluated a subset of the 64 grid points on (i) physics
532 stability under 5,000 random-action steps at $n \in \{2, 4, 8, 16, 32, 64\}$ joints per leg and (ii) trained
533 TD-MPC2 policy quality (10M-step preliminary runs) at the same DoF levels. Three distinct failure
534 regimes emerged across the grid, each driven by a different physical mechanism, and each ruling
535 out a contiguous region of the $(\alpha_{\text{range}}, \alpha_{\text{gear}})$ plane.

- 536 • **Regime 1 – low α_{range} + low α_{gear} (free joints + full per-joint torque): physics instability.** With $\alpha_{\text{range}} \leq 0.6$, the per-leg curvature budget grows as $n^{1-\alpha_{\text{range}}}$ – at $\alpha_{\text{range}}=0.3$, $n=32$
537 joints per leg the chain has $\approx 10\times$ the original cheetah’s worst-case bend budget, rendering
538 the legs into a spaghetti. Combined with $\alpha_{\text{gear}} \leq 0.3$ (per-joint motors retaining $\geq 70\%$
539 of baseline torque), random actions can drive multiple joints simultaneously to their limits
540 at high angular velocity. The constraint solver produces `mjWARN_BADQACC` crashes within
541 $\sim 1,000$ steps at $n \geq 32$. This regime is incompatible with stable simulation regardless of
542 policy training.
- 543 • **Regime 2 – moderate α_{range} + high α_{gear} (semi-free joints + weak motors): non-recoverable forward-leaning stumble.** At $\alpha_{\text{gear}} \in [0.4, 0.7]$ the per-joint hip gear at
544 $n=16$ joints per leg drops to $\approx 30\%$ – 50% of baseline. Combined with a permissive range
545 ($\alpha_{\text{range}} \in [0.5, 0.8]$, total curvature still allowed to grow as $n^{0.2}$ to $n^{0.5}$), the chain has more
546 freedom to deviate from upright than the weakened motors can correct. The trained policy
547 converges to a “forward-leaning stumble”: the cheetah leans into gravity, falls forward,
548 catches with the front legs, and repeats this motion as a low-reward absorbing pattern. The
549 front hip’s reduced extension authority is insufficient to push the torso back upright once it
550 has pitched forward, and even after 20M training steps the policy does not transition into a
551 true gallop.
- 552 • **Regime 3 – high α_{range} + high α_{gear} (tight joints + weak motors): no ground push-off at high DoF.** At $(\alpha_{\text{range}}, \alpha_{\text{gear}})=(1.0, 0.4-1.0)$ the curvature budget is exactly conserved (or
553 even shrunk) and the chain holds its shape passively, but the per-joint torque at the prox-
554 imal (hip) joint becomes too weak to overcome the cumulative inertial and gravitational
555 load of the $n-1$ distal segments below it. This is the well-known *serial-chain mechanical-*
556 *advantage loss*: in the original 3-joint cheetah a single hip torque rotates the leg as a near-
557 rigid pendulum; in the $n=32$ chain the same hip gear (now scaled to $\approx 9\%$ of baseline at
558 $\alpha_{\text{gear}}=1$) controls only the first segment, and the hip’s authority is absorbed by the immedi-
559 ate few flexing joints rather than reaching the foot to generate ground-reaction force. The
560 agent can stand and posture but cannot generate the concentrated impulse needed for stance
561 push-off; trained returns plateau at a low-velocity shuffle. This regime grows progressively
562 more severe with n , ruling out the entire upper-right corner of the sweep at high DoF even
563 when the simulator remains numerically stable.

Surviving region. Only $\alpha_{\text{range}}=1.0$ combined with $\alpha_{\text{gear}} \in [0, 0.1]$ avoids all three regimes simultaneously: range conservation rules out spaghetti Regime 1 by capping curvature, and prevents Regime 2 by leaving the chain insufficient freedom to enter the forward-lean basin; near-zero gear scaling rules out Regime 3 by preserving full proximal torque so the hip can still actuate the chain as a coordinated pendulum. We adopt $(\alpha_{\text{range}}, \alpha_{\text{gear}})=(1.0, 0.1)$, choosing $\alpha_{\text{gear}}=0.1$ over $\alpha_{\text{gear}}=0$ as a small hedge toward sub-linear total-torque growth at extreme n (justified below); $\alpha_{\text{gear}}=0$ is the proven walker recipe and would also be acceptable. Within the surviving region the $\alpha_{\text{range}}=1.0$ choice is essentially unique, providing full joint angle conservation.

B.3 Walker: Added Stiffness and Damping

The original DMControl walker uses very low damping (0.1) and no joint stiffness. At high DoF, subdivided legs become highly compliant serial chains that collapse under gravity before the policy can generate corrective forces. We add per-joint stiffness (profile: hip 200 $\rightarrow$ knee 100 $\rightarrow$ ankle 40) and damping (5.0 $\rightarrow$ 2.5 $\rightarrow$ 1.0) to all variants, including the 6-DoF base, so the physics are consistent across the entire DoF sweep.

This modification departs from the original DMControl walker dynamics. We make two arguments for its validity as a benchmark design:

1. **DoF-invariant modification.** The same stiffness and damping profile is applied to every DoF variant. The added stiffness/gear ratio (2.0) and damping/gear ratio (0.05) match the cheetah’s profiles, giving the walker consistent passive joint dynamics across the family. The modification changes the *absolute* difficulty relative to the original walker but does not introduce a *relative* advantage for any particular DoF count.
2. **Physical realism at high DoF.** Fully compliant serial chains do not arise in biological or robotic systems; springs and dampers are ubiquitous in joints. The added passive dynamics model realistic joint impedance and make the high-DoF variants physically interpretable.

We note that gear ratios are intentionally left unscaled in the walker, so total torque grows linearly with n . In combination with the fixed stiffness profile, this means higher-DoF walkers are strictly more powerful per limb than lower-DoF ones. Isolating DoF-specific difficulty from this torque confound is a limitation of the walker family; the cheetah’s near-constant torque budget ($\alpha = 0.1$) provides a cleaner control for this variable.

B.4 Finger: 64-DoF Specific Tuning

The 64-DoF finger receives additional per-variant tuning not applied to lower-DoF variants ($n \leq 32$):

- **Gear exponent shift.** For $n \leq 32$ the gear scales as $(2/n)^{0.7}$; at $n = 64$ we use $(2/n)^{0.85}$. The higher exponent more aggressively limits per-joint torque at extreme DoF, keeping the total torque budget closer to the $n \leq 32$ trend while providing enough torque for the fingertip (proximal joints) to overcome spinner friction.
- **Structural stiffness.** At $n = 64$, joint stiffness $k_s = 0.25 \cdot \text{gear}/\text{range_rad}$ is added; no stiffness is present for $n \leq 32$. Without it, the 64-joint chain buckles on contact with the spinner before it can impart angular momentum.
- **Spinner mass and friction scaling.** The spinner density is reduced by the same gear-scale factor, and the hinge frictionloss is similarly reduced, so the torque-to-inertia ratio faced by the fingertip matches that in the lower-DoF variants. Without this adjustment the 64-DoF finger cannot sustain rotation because it delivers many small impulses rather than one large hit, and the original spinner decelerates fully between contacts. See more details below.

These 64-DoF-specific adjustments mean the finger’s scaling law is not perfectly smooth across all DoF counts. We regard this as a practical trade-off: the alternative of keeping the spinner fixed and the gear law identical results in a 64-DoF agent that is essentially unable to learn the spin and turn tasks, making the point at that DoF uninformative for our comparative study. We attribute this effect to the weakening of proximal joints due to redistribution of torque across newer (and farther) joints. In our experiments, keep the setting same as in $n \leq 32$ gave us a weak 64 DoF finger that was unable to move the spinner.

B.5 Reacher: Design Details

The reacher is a planar serial-chain arm based on the DMControl reacher model. The base model has 2 DoF (shoulder and wrist hinges). Scaling extends the chain to 4, 8, 16, 32, and 64 DoF while holding the total arm length fixed at 0.24 m.

Joint ranges. The shoulder joint is left unlimited (free rotation), preserving the original reacher’s global-orientation degree of freedom. Non-shoulder joints distribute a total curvature budget of 160° equally, giving each non-shoulder joint a range of $\pm 160^\circ / (n-1)$.

Gear and passive dynamics. Motor gear is constant at 0.05 across all DoF counts (the XML default), so total torque per arm grows linearly with n . This is intentional: conserving total torque at this scale would leave individual joints too weak to reposition the arm quickly, and the task does not require force-intensive contact. Per-joint stiffness is set to $0.2 \cdot \text{gear}/\text{range_rad}$, which grows as n increases and per-joint ranges shrink, providing a passive restoring force that prevents the chain from curling under its own compliance. Damping is matched proportionally to stiffness with a fixed overdamped ratio to ensure numerical stability without oscillation.

B.6 Humanoid CMU: Design Details

The humanoid CMU is the most complex embodiment in HiGym, based on the 56-DoF CMU motion-capture skeleton from DMControl. Unlike the locomotion families, which scale by subdividing existing chains, the humanoid is extended to 80 DoF by inserting 24 intermediate bodies at the midpoints of the major limb segments (thigh, shin, foot, upper arm, forearm, and shoulder, bilaterally).

Joint splitting. Each intermediate joint inherits the rotation axes of its parent. The original joint ranges, gear ratios, and geom masses are split 50/50 between parent and child, exactly preserving the total curvature budget, torque budget, and body mass. Stiffness and damping classes are inherited from the parent joint. This 50/50 strategy differs from the pitch-only joint-splitting used in the locomotion families, reflecting the humanoid’s more heterogeneous joint structure where replacing the full range/gear of each joint is simpler and equally principled.

Scope. The humanoid is the only family for which HiGym provides a single scaled variant (80 DoF) rather than a sweep. Extending the joint-splitting procedure to higher DoF counts for this morphology is left as future work; the added complexity of whole-body balance limits the scaling analysis to a 56-versus-80-DoF comparison.

C Hyperparameters

Tables 5 and 4 list the hyperparameters used for **PPO** for readers to reproduce. TD-MPC2 and TD-M(PC)² are generally robust to hyperparameter tuning.

Table 4. Network architecture for PPO (actor and critic share the same MLP structure).

Component	Details
Actor (mean)	$\text{Linear}(d_{\text{obs}}, 256) \rightarrow \text{Tanh} \rightarrow \text{Linear}(256, 256) \rightarrow \text{Tanh} \rightarrow \text{Linear}(256, d_{\text{act}})$
Actor (std)	Learnable log-std parameter, initialized to 0, shared across states
Critic	$\text{Linear}(d_{\text{obs}}, 256) \rightarrow \text{Tanh} \rightarrow \text{Linear}(256, 256) \rightarrow \text{Tanh} \rightarrow \text{Linear}(256, 1)$
Initialization	Orthogonal weights ($\sigma = \sqrt{2}$); final actor layer $\sigma = 0.01$; biases = 0
Optimiser	Adam ($\epsilon = 10^{-5}$)

Table 5. Training hyperparameters for PPO.

Hyperparameter	Value	Description
<i>Rollout</i>		
num_envs	128	Parallel environments
num_steps	32	Steps per rollout per environment
batch_size	4,096	$= \text{num_envs} \times \text{num_steps}$
num_minibatches	32	Mini-batches per update
minibatch_size	128	$= \text{batch_size} / \text{num_minibatches}$
total_timesteps	50,000,000	Total environment steps
<i>optimization</i>		
learning_rate	1×10^{-3}	Adam learning rate
anneal_lr	True	Linearly decay LR to 0
update_epochs	8	PPO epochs per iteration
max_grad_norm	1.0	Gradient clipping norm
<i>PPO objectives</i>		
clip_coef	0.2	PPO clipping ϵ
clip_vloss	True	Clip value-function loss
ent_coef	0.005	Entropy bonus coefficient
vf_coef	0.5	Value-function loss coefficient
target_kl	0.02	Early-stop KL threshold
norm_adv	True	normalize advantages per mini-batch
<i>Return estimation</i>		
gamma	0.99	Discount factor
gae_lambda	0.95	GAE λ
reward_scaler	1.0	Reward scaling factor
<i>Evaluation</i>		
eval_interval	1,000,000	Timesteps between evaluations
eval_episodes	32	Episodes averaged per evaluation
eval_num_envs	32	Parallel envs during evaluation
eval_deterministic	True	Use mean action (no sampling)

D Extended Empirical Results

D.1 Sample Efficiency vs. DoF

Figure 12 complements the per-DoF learning curves with a single sample-efficiency metric: for each (family, task, seed, algorithm), we record the first environment step at which a 5-point rolling mean of the normalized return crosses 75% of the best observed return on that task; values are aggregated within each family via IQM, and runs that never reach the threshold within the step budget are drawn as rightward “did-not-solve” arrows at the budget ceiling. PPO exhibits the steepest scaling in every locomotion family and fails to reach the threshold altogether at high DoF, while TD-MPC2 begins to require markedly more steps as DoF crosses ~ 64 . TD-M(PC)² solves strictly more cells within the budget, particularly in Cheetah and Quadraped at 128–256 DoF, where TD-MPC2 itself starts to falter.

D.2 Extended Learning Curves

Figure 20 extends the per-DoF learning curves of Figure 8 to all six HiGym environment families and all tested DoF variants, providing complete coverage of training dynamics across the benchmark. Each panel shows evaluation return versus environment steps for PPO, TD-MPC2, and TD-M(PC)²; individual seeds are shown as min-max bands with the inter-seed mean bolded. The full grid confirms the patterns observed in the main text: PPO’s flat trajectories at high DoF are a universal failure mode in locomotion; TD-MPC2 retains learning ability but with increasing variance at high DoF; TD-M(PC)² provides the most consistent improvement across families and dimensionalities.

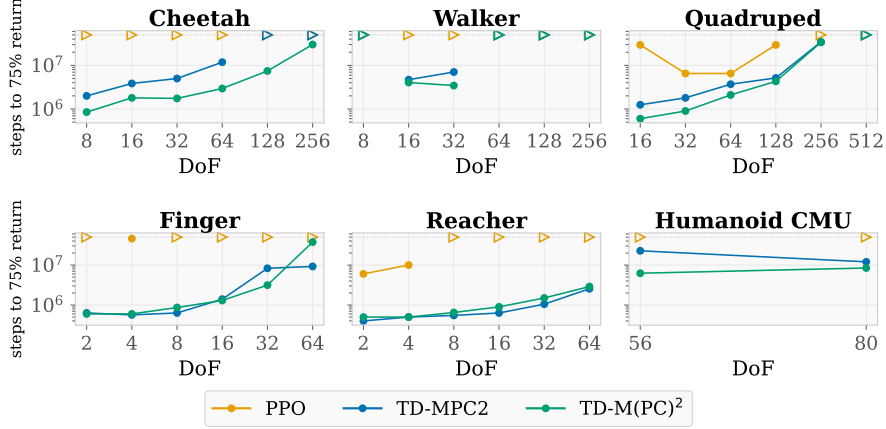


Figure 12. **Sample efficiency versus DoF.** Training steps required to first reach 75% of best-observed return on each task, aggregated across tasks within a family via IQM. The x-axis is DoF on a log-2 scale; the y-axis is environment steps on a log scale. Cells that never reach the threshold within the step budget are shown as rightward “did-not-solve” arrows at the budget ceiling.

670 D.3 Performance Profile

671 Figure 13 plots the performance profile [Agarwal et al., 2021]: the fraction of (task, DoF, seed)
 672 runs whose asymptotic IQM meets or exceeds each normalized threshold $\tau \in [0, 1]$. TD-M(PC)²
 673 lies strictly above TD-MPC2 at every threshold, and both model-based methods lie well above PPO
 674 throughout. The separation is largest at intermediate thresholds ($\tau \approx 0.4$ – 0.8), where most PPO
 675 runs stall, while both TD-MPC2 variants continue accumulating high-performing runs, with the gap
 676 widening beyond $\tau = 0.8$.

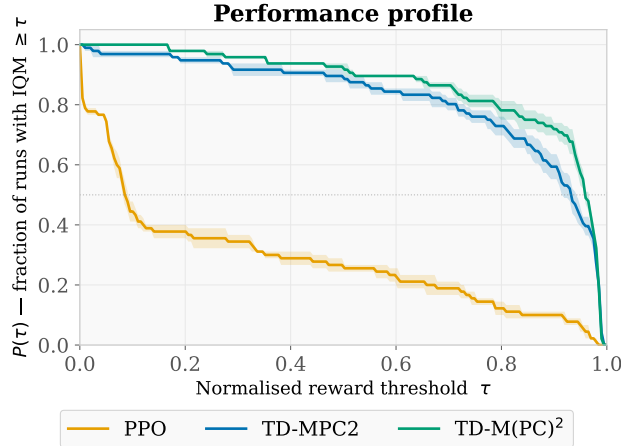


Figure 13. **Performance profile across HiGym.** Fraction of (task, DoF, seed) runs with asymptotic IQM $\geq \tau$ for each $\tau \in [0, 1]$; shaded bands are stratified-bootstrap 95% CIs. TD-M(PC)² stochastically dominates TD-MPC2 at every threshold, and both model-based methods dominate PPO.

677 D.4 Algorithm Ranking by DoF

678 Figure 14 shows the mean rank of each algorithm at each DoF, averaged across all (family, task)
 679 cells that share that DoF. Bootstrap 95% confidence intervals are obtained by resampling cells with
 680 replacement. PPO holds a near-constant rank 3 throughout the DoF range. TD-MPC2 starts near
 681 rank 1.5 at low DoF but degrades toward rank 2.5 at 128–256 DoF as value overestimation intensi-
 682 fies. TD-M(PC)² improves relative to TD-MPC2 at high DoF, consistent with the policy-constraint
 683 loss suppressing the overestimation failure mode described in Section A.2. The per-embodiment
 684 breakdown in Figure 7 (main text) shows this pattern is strongest in locomotion families.

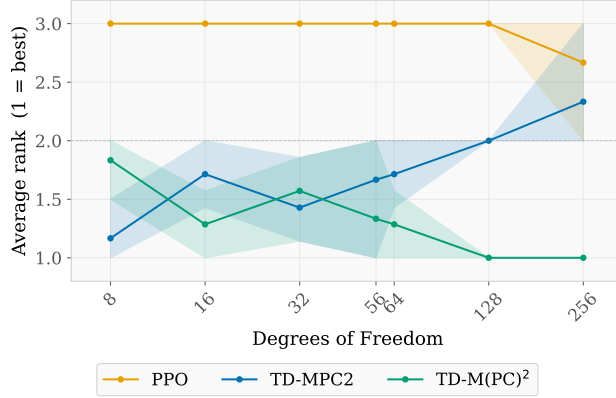


Figure 14. **Mean algorithm rank vs. DoF.** Rank averaged across all (family, task) cells at each DoF (DoF 8–256, excluding 80 which only appears for Humanoid CMU); 95% stratified-bootstrap CIs. Rank 1 is best. n values indicate the number of cells contributing at each DoF.

685 D.5 Aggregate Performance Statistics

686 Figure 15 presents a complementary aggregate view of benchmark performance following reliable
687 conventions [Agarwal et al., 2021]. The figure displays a per-cell ranking heatmap: each row is a
688 (task, DoF) cell and each column an algorithm, color-coded from green (rank 1) to red (rank 3).
689 TD-M(PC)² dominates the high-DoF locomotion cells while PPO wins only the lowest-DoF ma-
690 nipulation cells. Specifically, we see TD-M(PC)² as a clear winner across Cheetah, Quadruped and
691 Humanoid CMU tasks.

692 D.6 Pairwise Statistical Comparison

693 Figure 16 reports the pairwise probability of improvement [Agarwal et al., 2021] for each ordered
694 algorithm pair. All three bars have lower confidence bounds strictly above 0.5, confirming the
695 ordering TD-M(PC)² > TD-MPC2 > PPO is statistically robust at the benchmark level. The TD-
696 M(PC)² vs. PPO and TD-MPC2 vs. PPO bars are both near $P \approx 0.9$, reflecting the large and
697 consistent margin of model-based over model-free methods. The TD-M(PC)² vs. TD-MPC2 bar is
698 narrower ($P \approx 0.53$), reflecting that the PC-loss benefit is concentrated at high DoF and thus diluted
699 when averaging across the full benchmark including low-DoF cells where both variants perform
700 similarly.

701 D.7 Planning Budget Sensitivity

702 Figures 17 and 18 extend the representative panels of Figure 9 to all DoF variants of Walker (8–
703 256 DoF) and Quadruped (16–512 DoF). Each figure is arranged as a 3×4 grid: rows correspond to
704 DoF pairs (low, mid, high) and within each row the four panels show, for each of the two DoFs, the
705 horizon sweep (CEM iterations fixed at 6) and the CEM iterations sweep (horizon fixed at 3). Col-
706 ored ★ marks each algorithm’s default operating point (horizon= 3, CEM= 6) used in all training
707 runs; colored dashed lines show the corresponding policy-only baselines π_θ (no MPC planning) for
708 each algorithm.

709 Across both families, TD-M(PC)² exhibits consistently flatter curves than TD-MPC2, with the gap
710 most pronounced at high DoF. At low-to-mid DoF (Walker-8 through 64, Quadruped-16 through
711 64), both methods reach comparable performance at the default operating point, but TD-MPC2
712 shows steeper sensitivity to short horizons and few CEM iterations, reflecting its greater reliance
713 on the planner to compensate for policy drift. At high DoF (Walker-128/256, Quadruped-128–
714 512), the PC advantage in absolute performance is largest and budget robustness is most striking:
715 TD-M(PC)² curves remain near-flat while TD-MPC2 is highly sensitive or already operating near
716 floor performance. Notably, the per-algorithm policy baselines reveal a second dimension of this
717 advantage: at high DoF, TD-M(PC)²’s policy π_θ alone achieves substantial return while TD-MPC2’s
718 policy collapses to near zero, confirming that the PC loss trains a qualitatively stronger actor that

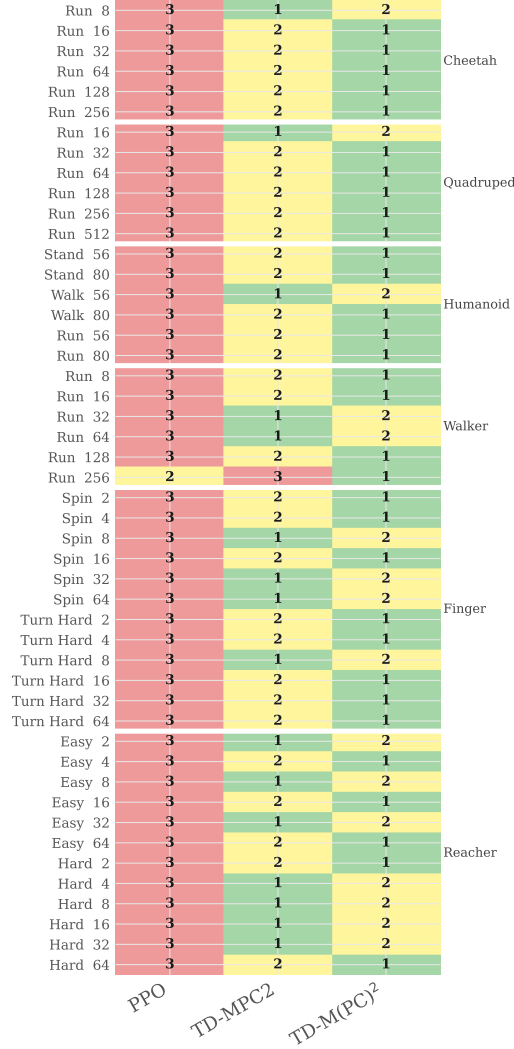


Figure 15. **Benchmark-level aggregate performance.** Per-cell ranking heatmap showing each algorithm’s rank (1 = best) on every (task, DoF) cell.

partially compensates for planning budget reductions. Note that Quadruped-512 results fail to learn any useful behavior as the base reward starts from 500.

D.8 Curriculum Learning from Low-DoF Checkpoints

A natural question raised by the HiGym scaling setup is whether representations learned on a low-DoF variant can accelerate training on a higher-DoF variant of the same family. Because the joint-splitting procedure preserves task semantics, morphological structure, and observation semantics across DoF counts, the majority of TD-MPC2’s learned components, i.e., the encoder h , the hidden layers of the dynamics model d , the reward head R , the Q-function hidden layers, and the policy hidden layers π have identical shapes regardless of action dimension. Only the layers that directly interface with the action vector need to be re-initialized when transferring to a different DoF count (Table 6).

Protocol. We take a TD-M(PC)² checkpoint trained on cheetah-8-run for 6M steps and transfer it to cheetah-256-run via partial checkpoint loading (`partial_checkpoint=true`). The trainer calls `load_state_dict(strict=False)` with the subset of shape-compatible tensors and freshly

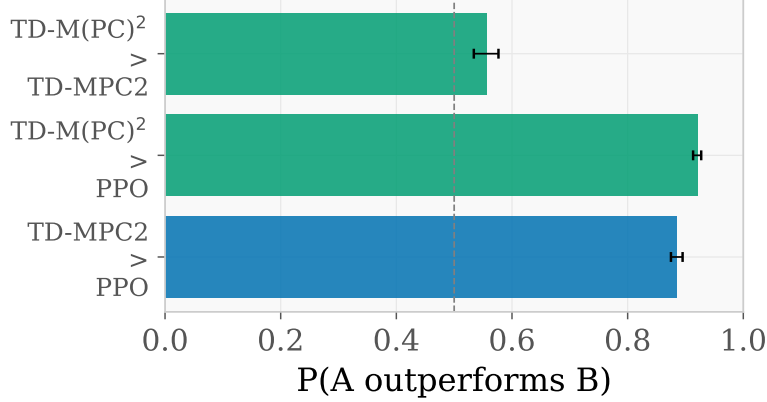


Figure 16. **Pairwise probability of improvement.** For each ordered pair of algorithms (A, B) , the probability $P(A \succ B)$ on a uniformly-sampled (task, DoF, seed) triple, estimated via the Mann-Whitney U statistic [Agarwal et al., 2021]. Error bars denote 95% confidence intervals. The ordering $\text{TD-M(PC)}^2 \succ \text{TD-MPC2} \succ \text{PPO}$ is statistically maintained at the benchmark level.

Table 6. **Layer-wise transferability from cheetah-8-run to cheetah-256-run.** Shapes are (out, in); latent_dim= 512, mlp_dim= 512.

Layer	Shape (8-DoF)	Shape (256-DoF)	Transfer?
Encoder h (all)	action-independent	same	✓
Policy hidden $\pi_{[0,1]}$	(512, 512)	same	✓
Policy output $\pi_{[2]}$	(16, 512)	(512, 512)	× re-init
Dynamics input $d_{[0]}$	(512, 520)	(512, 768)	× re-init
Dynamics hidden $d_{[1,2]}$	(512, 512)/(512, 512)	same	✓
Reward input $R_{[0]}$	(512, 520)	(512, 768)	× re-init
Reward hidden $R_{[1,2]}$	(512, 512)/(512, 101)	same	✓
Q-function input $\times 5$	(512, 520)	(512, 768)	× re-init
Q-function hidden $\times 5$	(512, 512)/(512, 101)	same	✓

733 re-initialises the action-interfacing layers listed in Table 6. The training budget (50M env steps) and
734 all hyperparameters are identical to the from-scratch runs in our main experiments.

735 **Results.** Figure 19 compares the transferred TD-M(PC)^2 run against from-scratch TD-MPC2 and
736 TD-M(PC)^2 baselines at 256 DoF. The warm-started policy reaches a normalized reward of ≈ 0.75
737 within the first $\approx 10\text{M}$ steps, a regime in which both from-scratch baselines are still below 0.5.
738 Across the first $\approx 40\text{M}$ steps the transferred run sits clearly above the from-scratch TD-M(PC)^2
739 mean and far above from-scratch TD-MPC2, which remains highly volatile throughout training.
740 The gap narrows in the second half of training as the from-scratch TD-M(PC)^2 baseline catches up,
741 indicating that the warm-start primarily provides a substantial sample-efficiency benefit rather than
742 a higher asymptotic ceiling. This is consistent with the interpretation that the encoder, dynamics,
743 and reward representations learned at 8 DoF transfer usefully to the 256 DoF variant, leaving only
744 the action-interfacing layers to be learned from scratch.

745 **Caveat and outlook.** We emphasise that the experiment above is a single proof-of-concept run
746 (one embodiment, one target DoF, one source checkpoint, one seed for the transferred policy) and
747 is not intended to support a quantitative claim about the magnitude or generality of the effect. A
748 more thorough study by sweeping source/target DoF pairs, embodiments, and seeds, and isolating
749 which world-model components contribute most to the transferred sample-efficiency gain, is left as
750 a promising direction for future work.

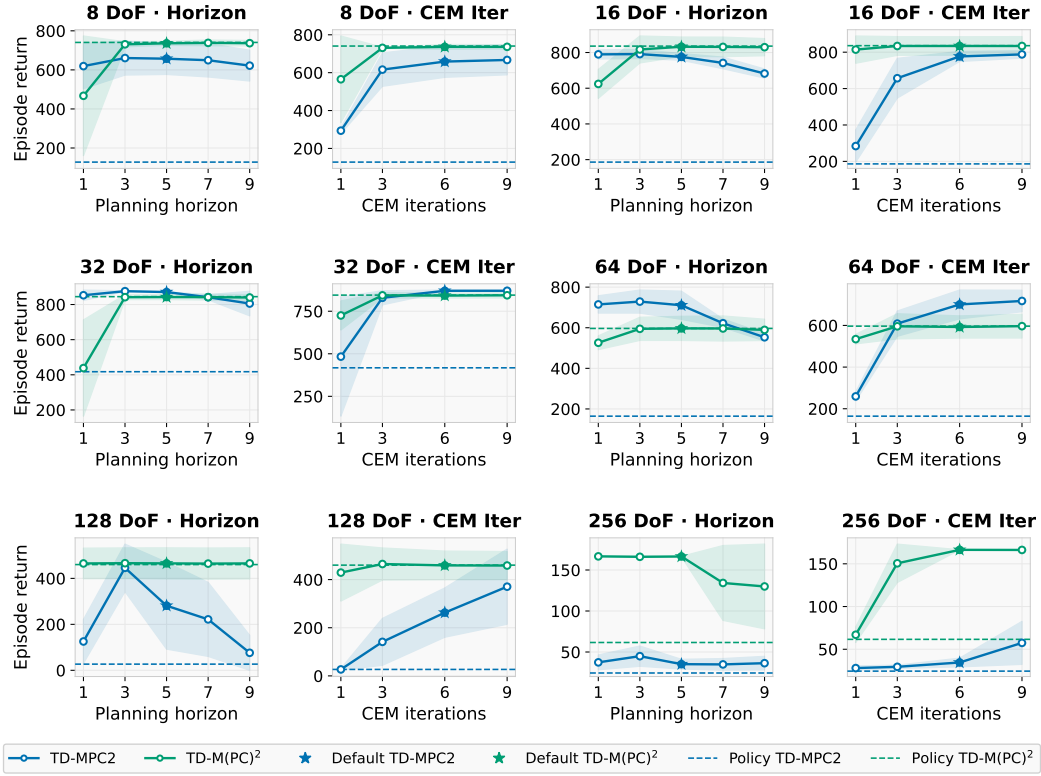


Figure 17. **Full planning budget sensitivity for Walker (8–256 DoF).** 3×4 grid: rows pair DoFs (8 & 16; 32 & 64; 128 & 256); within each row the four panels show horizon sweep then CEM iterations sweep for each DoF. CEM iterations fixed at 6 for horizon panels; horizon fixed at 3 for CEM panels. Colored $\star$ marks each algorithm’s default operating point; colored dashed lines show each algorithm’s policy-only baseline π_θ (no planning).

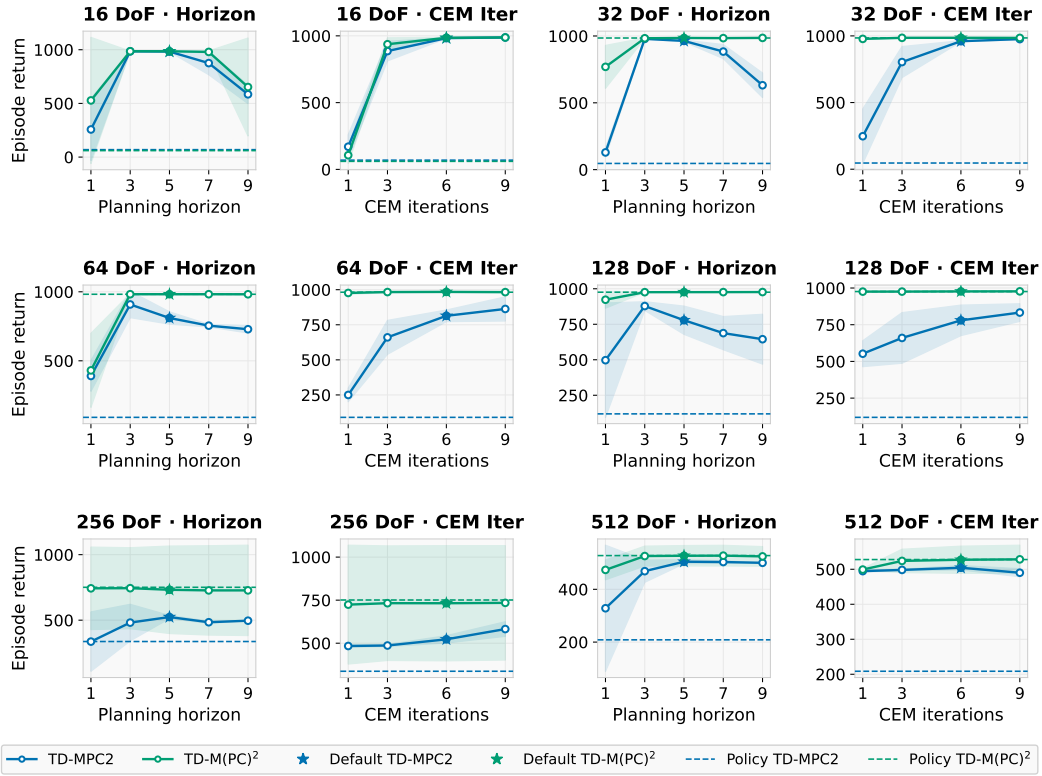


Figure 18. Full planning budget sensitivity for Quadraped (16–512 DoF). Layout as Figure 17. Colored ★ marks each algorithm’s default operating point; colored dashed lines show each algorithm’s policy-only baseline π_θ (no planning). Only Quadraped-512 fails to learn any useful behavior.

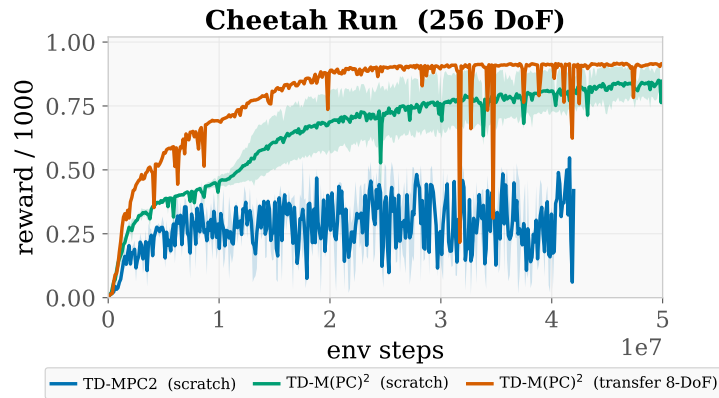


Figure 19. Curriculum learning: cheetah-8-run → cheetah-256-run. Training curves at 256 DoF. The transferred TD-M(PC)² policy (orange), warm-started from a 6M-step cheetah-8-run checkpoint via partial checkpoint loading, reaches near-asymptotic reward within the first ≈ 15 M steps, well ahead of from-scratch TD-M(PC)² (green) and from-scratch TD-MPC2 (blue). Individual seeds are presented as min-max bands.

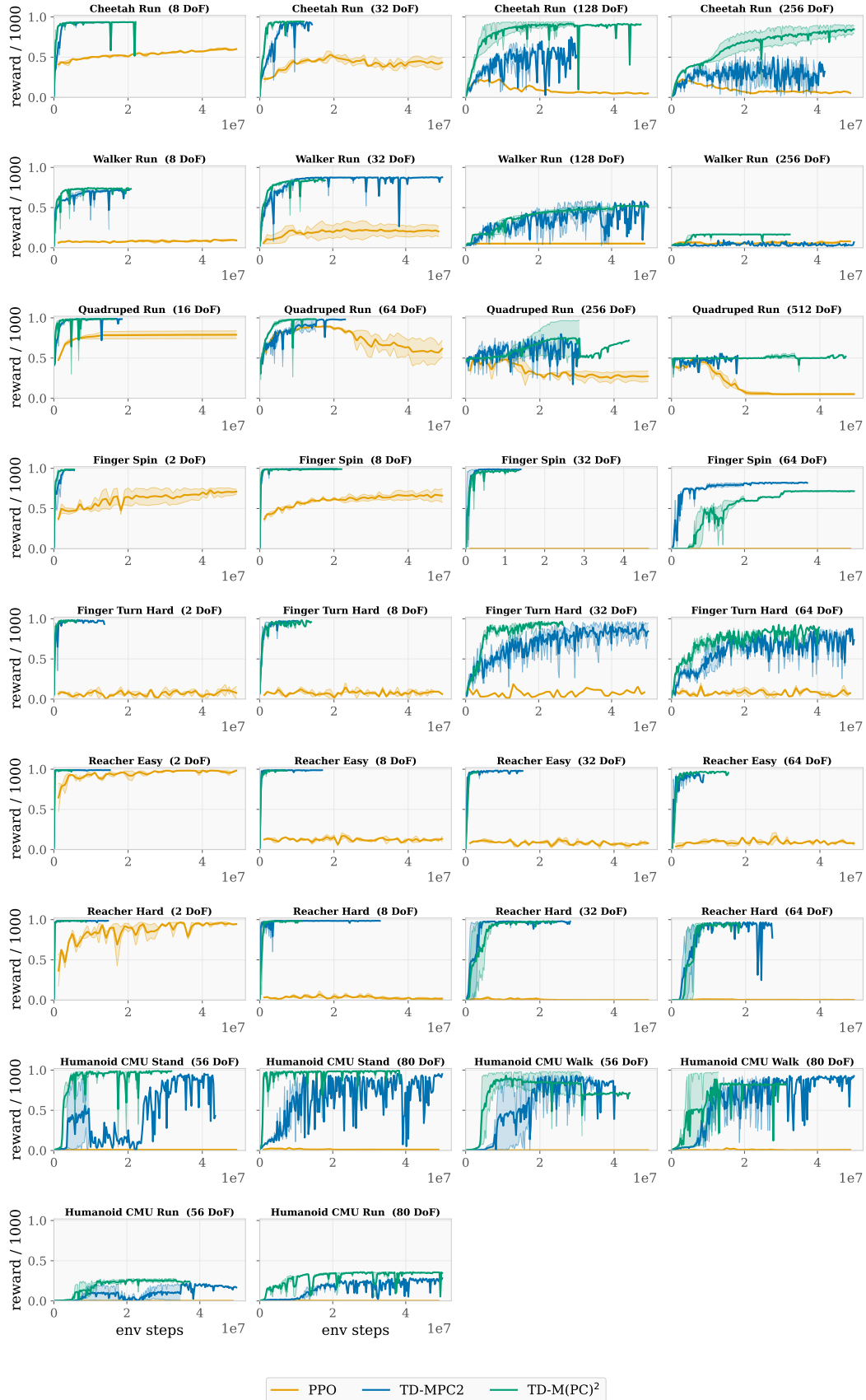


Figure 20. **Full learning curves grid.** Per-seed training return for all (family, task, DoF) cells in HiGym. Rows correspond to environment families and tasks; columns index DoF variants. Faint traces are individual seeds; bold lines are inter-seed means.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: We propose a benchmark for continuous control with high-dimensional action spaces. Our benchmark is specifically designed to isolate action space scaling by keeping tasks and dynamics as similar as possible, allowing us to study scaling independent of other factors. Our main experimental claim is that model-based methods such as TD-MPC2 degrade substantially less as dimensionality increases compared to PPO, a model-free baseline. Our benchmark results throughout the paper support this claim.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: We discuss limitations in Section 6.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: No theoretical results

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide implementation details in Appendix C and also release our code.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Our code is made available at <https://higym.github.io>.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: We detail this in Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We run experiments across multiple embodiments, tasks, DoFs, and random seeds, and report 95% percentile-bootstrap CIs (10,000 resamples) over per-run asymptotic IQM in most figures, unless mentioned otherwise (some experiments use 95% stratified-bootstrap CIs with seeds as bootstrap unit).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We discuss this in Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have carefully reviewed the NeurIPS Code of Ethics and believe that our research conforms to this code.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: We do not believe that there is any immediate societal impact of our work. However, we acknowledge that this is a fast-moving field and that the societal consequences of any particular paper can be hard to predict.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We believe that the risk of misuse is small.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We base our benchmark on DMControl [Tassa et al., 2018] which we cite. DMControl has a permissive Apache 2.0 license.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.

- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

1022 13. New assets

1023 Question: Are new assets introduced in the paper well documented and is the documenta-
1024 tion provided alongside the assets?

1025 Answer: [Yes]

1026 Justification: We provide documentation on our webpage as well as in the codebase itself.

1027 Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

1036 14. Crowdsourcing and research with human subjects

1037 Question: For crowdsourcing experiments and research with human subjects, does the pa-
1038 per include the full text of instructions given to participants and screenshots, if applicable,
1039 as well as details about compensation (if any)?

1040 Answer: [N/A]

1041 Justification: Not applicable.

1042 Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

1051 15. Institutional review board (IRB) approvals or equivalent for research with human 1052 subjects

1053 Question: Does the paper describe potential risks incurred by study participants, whether
1054 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1055 approvals (or an equivalent approval/review based on the requirements of your country or
1056 institution) were obtained?

1057 Answer: [N/A]

1058 Justification: Not applicable.

1059 Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- 1062 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1063 may be required for any human subjects research. If you obtained IRB approval,
1064 you should clearly state this in the paper.
- 1065 • We recognize that the procedures for this may vary significantly between institutions
1066 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1067 guidelines for their institution.
- 1068 • For initial submissions, do not include any information that would break anonymity
1069 (if applicable), such as the institution conducting the review.

1070 16. Declaration of LLM usage

1071 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1072 non-standard component of the core methods in this research? Note that if the LLM is used
1073 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1074 scientific rigor, or originality of the research, declaration is not required.

1075 Answer: [N/A]

1076 Justification: The core method development in this work does not involve LLMs as any
1077 important, original, or non-standard components.

1078 Guidelines:

- 1079 • The answer [N/A] means that the core method development in this research does not
1080 involve LLMs as any important, original, or non-standard components.
- 1081 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
1082 not be described.